

MECHATRONICS SYSTEM INTEGRATION

MCTA 3203

SEMESTER 2, 2025/2026

SECTION 1

WEEK 3: SERVO MOTOR

(SERIAL COMMUNICATION BETWEEN ARDUINO AND PHYTON)

INSTRUCTOR:

ASSOC. PROF. EUR. ING. IR. TS. GS. INV. DR ZULKIFLI BIN ZAINAL ABIDIN

GROUP 3

NO.	NAME	MATRIC NUM
1.	AMIR AIMAN BIN ARIFF AFFENDI	2315925
2.	ANIS AQILAH BINTI ABDULLAH	2329678
3.	AZRIL ULWAN BIN MOHAMMAD IMRI	2314617

DATE OF EXPERIMENT:

11 MARCH 2026

ABSTRACT

This experiment demonstrates serial communication between an Arduino and a computer to control a servo motor. The objective was to transmit control signals from a Python interface to the Arduino, enabling the servo motor to rotate according to user-defined angles or real-time potentiometer input. An LED indicator was also integrated to provide visual feedback based on the servo position, while real-time potentiometer data was plotted using Python's matplotlib library. The experiment successfully illustrates how serial communication can be utilized to achieve interactive control between software and hardware components, which is essential in mechatronic system integration.

TABLE OF CONTENTS	3
1.0 INTRODUCTION	4
EXPERIMENT 1: READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION	
2.0 MATERIALS AND EQUIPMENTS	5
3.0 EXPERIMENTAL SETUP	6
4.0 METHODOLOGY	7
5.0 DATA COLLECTION	11
6.0 DATA ANALYSIS	12
7.0 RESULTS	14
EXPERIMENT 2: CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION	
2.0 MATERIALS AND EQUIPMENTS	16
3.0 EXPERIMENTAL SETUP	17
4.0 METHODOLOGY	18
5.0 DATA COLLECTION	23
6.0 DATA ANALYSIS	24
7.0 RESULTS	25
8.0 DISCUSSION	27
9.0 CONCLUSION	29
10.0 RECOMMENDATIONS	30
11.0 REFERENCES	31
APPENDICES	32
ACKNOWLEDGEMENTS	33
STUDENT'S DECLARATION	34

1.0 INTRODUCTION

This study investigates the bidirectional capabilities of serial communication by interfacing an Arduino microcontroller with Python. The research is divided into two functional phases: the first focuses on data acquisition by streaming analog potentiometer readings to a computer, while the second explores remote actuation by commanding a servo motor's position through Python based serial inputs. By utilizing Python as a high-level processing engine and Arduino as the hardware interface, the experiment demonstrates how complex mechatronic systems can be monitored and controlled from an external software environment.

The technical framework of the experiment rests on precise synchronization of hardware and software protocols. Establishing a reliable link requires aligning the baud rate and implementing robust data encoding methods to ensure that signals are not lost or corrupted during transit. This process leverages specialized libraries specifically *Servo.h* for managing Pulse Width Modulation (PWM) on the Arduino and *PySerial* for managing the data stream in Python to bridge the gap between digital logic and physical motion.

The central hypothesis is that a stable serial connection will facilitate real-time accuracy in both reading sensor data and executing motor commands. A successful outcome would confirm that serial communication is an efficient and dependable medium for low-latency feedback loops. Ultimately, this experiment serves to validate a fundamental architecture used in modern robotics and industrial automation, where centralized processing power is used to orchestrate decentralized hardware components.

EXPERIMENT 1: READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION

2.0 MATERIALS AND EQUIPMENTS

Here is the list of all equipment used in the experiment:

1. Arduino Uno board
2. Potentiometer
3. Breadboard
4. Jumper wires
5. LED
6. 220 Ω resistor
7. USB cable
8. Computer with Arduino IDE and Python

3.0 EXPERIMENTAL SETUP

The experiment setup consists of an Arduino Uno connected to a potentiometer and an LED:

- The potentiometer's first terminal was connected to 5V, the second terminal to GND, and the center (wiper) terminal to analog input pin A0.
- An LED was connected to digital pin 9 through a 220 Ω resistor, which limited current to prevent LED damage.
- The Python program on the computer receives potentiometer readings from the Arduino through the serial port (USB connection) and displays them on the terminal.

The connection allowed the potentiometer's analog readings to be transmitted from the Arduino to the computer and for the LED to act as a visual indicator when the threshold value was exceeded.

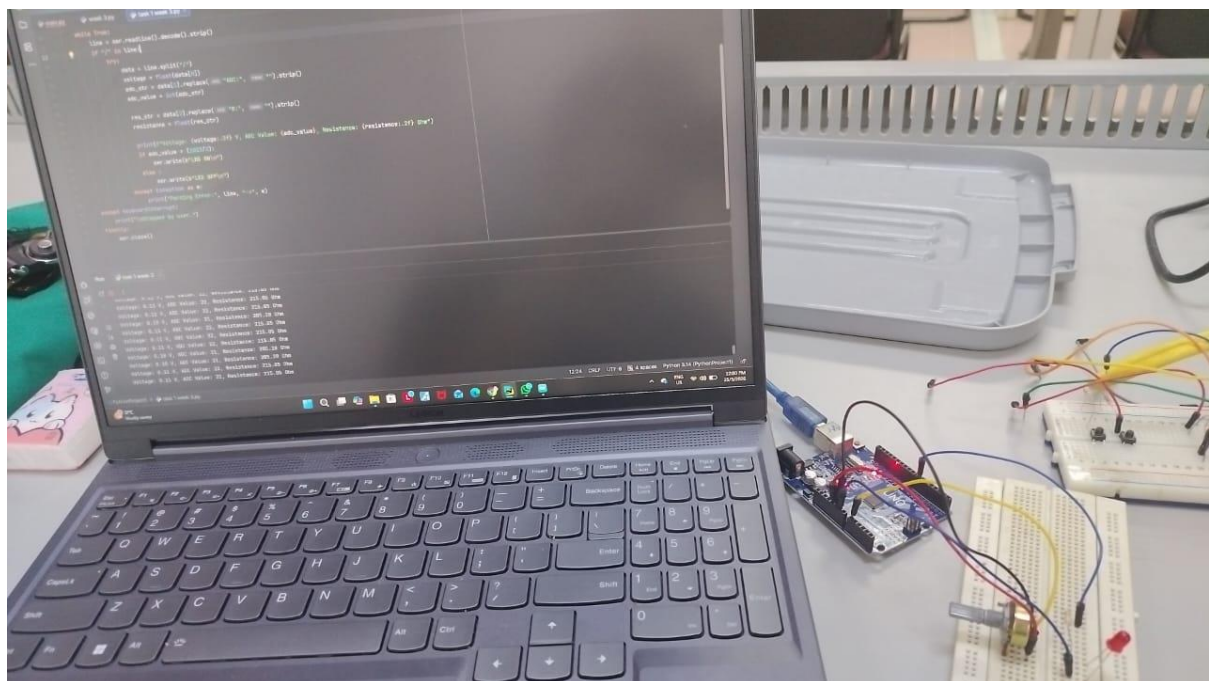


Figure 3.1 Circuit diagram for Task 1

4.0 METHODOLOGY

1. Circuit Construction:

The circuit was assembled on a breadboard according to the configuration specified in the experimental setup. The potentiometer was interfaced with the Arduino Uno by connecting one outer pin to the 5V supply, the opposite outer pin to GND, and the middle wiper pin to the analog input pin A0. An LED was also connected to digital pin 9 through a 220 Ω current-limiting resistor to ensure safe operation. The Arduino Uno was then linked to the computer using a USB cable, which provided both power to the circuit and serial communication for program uploading and monitoring.

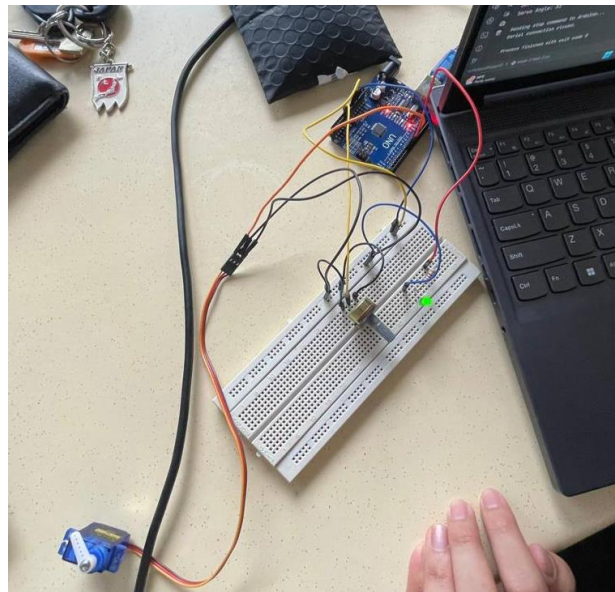


Figure 4.0: Circuit Connection for Task 1

2. Programming:

The Arduino IDE was utilized to upload a C++ program that continuously acquired analog signals from the potentiometer and sent the readings to the serial monitor. Serial communication was configured at a baud rate of 9600, and the analog values from pin A0 were obtained using the `analogRead()` function. The program also implemented conditional logic to

control the LED operation, where the LED switched ON when the potentiometer value exceeded 512, which is approximately half of the analog input range, and switched OFF when the value was below this threshold. In addition, a Python script developed using the pyserial library was used to receive and display the transmitted potentiometer readings in real time through the computer terminal.

Arduino Coding:

```
int potPin = A0;
int ledPin = 9; // or any digital pin through a 220  $\Omega$  resistor
int potValue = 0;

void setup() {
  Serial.begin(9600);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  potValue = analogRead(potPin);

  // LED ON if potentiometer > half
  if (potValue > 512) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }

  // Send potentiometer reading to Python
  Serial.println(potValue);

  delay(100); // send data every 0.1 s
}
```

Phyton Coding:

```
import serial
import time

ser = serial.Serial('COM11', 9600)
time.sleep(2)

print("Running.. Press CTRL+C to stop.")

try:
    while True:
        line = ser.readline().decode().strip()
        if "/" in line:
            try:
                data = line.split("/")
                voltage = float(data[0])
                adc_str = data[1].replace("ADC:", "").strip()
                adc_value = int(adc_str)

                res_str = data[2].replace("R:", "").strip()
                resistance = float(res_str)

                print(f'Voltage: {voltage:.2f} V, ADC Value: {adc_value}, Resistance:
{resistance:.2f} Ohm")
                if adc_value > (1023/2):
                    ser.write(b"LED ON\n")
                else :
                    ser.write(b"LED OFF\n")
            except Exception as e:
                print("Parsing Error:", line, "->", e)
except KeyboardInterrupt:
    print("\nStopped by user.")
finally:
    ser.close()
```

3. Operation:

After the Arduino program was successfully uploaded, the system was powered through the USB connection between the Arduino and the computer. The Python script was then executed to establish serial communication with the Arduino board. When the potentiometer knob was adjusted, the resistance of the potentiometer changed, causing variations in the analog voltage supplied to analog pin A0. The Arduino converted the analog voltage into corresponding digital values ranging from 0 to 1023 and transmitted the data to the Python application for real-time display.

4. Observation

As the potentiometer was rotated clockwise, the analog readings shown on the serial monitor gradually increased, and the LED was activated once the reading exceeded approximately 512. Conversely, rotating the potentiometer counterclockwise reduced the analog readings, causing the LED to switch OFF when the value dropped below the threshold. The Python terminal continuously displayed the updated potentiometer values, while the Arduino Serial Plotter provided a graphical waveform representation of the changing readings.

5. Code Behaviour

The Arduino program utilized the `analogRead(A0)` function to obtain analog voltage readings from the potentiometer and used `Serial.println()` to transmit the data to the computer through serial communication. A `delay()` function was incorporated to regulate the rate of data transmission and ensure stable communication. On the computer side, the Python script employed the `ser.readline().decode().strip()` command to receive, decode, and display the incoming serial data from the Arduino. Both the Arduino and Python programs operated at a matching baud rate of 9600 to maintain synchronized and reliable serial communication. This

setup successfully demonstrated one-way serial communication between the Arduino and Python system, integrating analog sensor input, real-time data visualization, and LED control.

5.0 DATA COLLECTION

In this experiment, data acquisition was carried out using a potentiometer interfaced with the Arduino Uno through analog input pin A0. The potentiometer functioned as a variable resistor that transformed rotational movement into varying voltage levels, which were interpreted by the Arduino as analog readings within the range of 0 to 1023, representing an input voltage range of 0 to 5 V.

The Arduino continuously transmitted these analog readings to the computer via USB serial communication using the `Serial.println()` function. On the computer side, the transmitted data were received and displayed in real time using a Python program developed with the PySerial library. Furthermore, the Arduino Serial Plotter was used to graphically visualize the changing potentiometer values dynamically as the knob position was adjusted.

This setup enabled continuous and accurate observation of the relationship between the potentiometer rotation, resistance variation, and corresponding voltage changes. The following table summarizes the collected data and the instruments used during the data acquisition process.

Data Type	Instrument/Sensor	Signal Range	Output Form	Purpose
Analog voltage reading	Potentiometer	0–5 V (0–1023 ADC units)	Serial data output	To measure and transmit analog voltage readings to Python

Serial data	Arduino–Python link	Digital serial stream	Displayed as numeric output in Python terminal	To observe and record real-time sensor readings
-------------	---------------------	-----------------------	--	---

The recorded data demonstrated a consistent and gradual change in potentiometer readings as the knob position was adjusted. The results verified that the Arduino successfully detected variations in the analog voltage and reliably transmitted the data to the Python program for real-time monitoring and analysis.

6.0 DATA ANALYSIS

The experiment data were evaluated to examine the behavior of the potentiometer and the reliability of data transmission between the Arduino Uno and the Python interface. As the potentiometer shaft rotated, the voltage supplied to analog pin A0 changed continuously. The Arduino's ADC module converted these analog voltages into digital readings within the range of 0 to 1023. The relationship between the measured ADC value and the corresponding voltage is given by:

$$V_{out} = \frac{\text{ADC Value}}{1023} \times 5$$

This equation allows the analog voltage to be estimated directly from the digital reading. For instance, when the ADC value approached 512, the calculated voltage was approximately 2.5 V, indicating that the potentiometer was adjusted near its center position.

Throughout the experiment, the readings observed from the Python terminal and Serial Plotter changed progressively according to the knob rotation. The transmitted values remained stable and responsive, showing that the serial communication process operated efficiently without

noticeable lag or missing data packets. The communication speed of 9600 baud was sufficient to provide smooth real-time monitoring of the sensor output.

From the obtained results, it was observed that increasing the potentiometer rotation increased the ADC reading proportionally, while rotating it in the opposite direction reduced the reading accordingly. This verified the linear response characteristic of the potentiometer and confirmed that the Arduino and Python systems were able to communicate and process real-time analog data successfully.

7.0 RESULTS

The experiment successfully demonstrated that the potentiometer's analog voltage output could be read, converted, and transmitted from the Arduino to Python via serial communication. The observed data confirmed that the potentiometer readings increased and decreased proportionally with the rotation of the knob, following a linear trend between 0 and 1023 digital counts.

Table 7.1 summarizes the typical readings obtained from the potentiometer during various knob positions. These readings were displayed in the Python terminal and visualized in the Arduino Serial Plotter, showing consistent and real-time data transmission without communication delay.

Knob Position	Analog Reading (0–1023)	Calculated Voltage (V)	Observation
Minimum (0°)	0	0.00	No output voltage detected
Quarter Turn	256	1.25	Low voltage, LED off
Midpoint (Half Turn)	512	2.50	Medium voltage, LED threshold range
Three-Quarter Turn	768	3.75	High voltage output

Maximum (Full Turn)	1023	5.00	Maximum voltage detected
------------------------	------	------	-----------------------------

Table 7.1 Sample Potentiometer Readings

Overall, the findings verified that the Arduino and Python communicated effectively, providing accurate and stable real-time monitoring of analog input signals. This validates the functionality of serial communication for continuous data acquisition applications.

EXPERIMENT 2: CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION

2.0 MATERIALS AND EQUIPMENTS

1. Arduino Uno board
2. Servo motor
3. Potentiometer
4. Breadboard
5. Jumper wires
6. LED
7. 220 Ω resistor
8. USB cable
9. Computer with Arduino IDE and Python

3.0 EXPERIMENTAL SETUP

This setup connects a servo motor to the Arduino and uses a potentiometer as a real-time control input.

- The servo motor was connected such that its signal pin was attached to digital pin 11, its VCC to 5V, and GND to GND on the Arduino.
- The potentiometer was used as an analog input device, with one terminal connected to 5V, another to GND, and the middle terminal (wiper) connected to analog input pin A0.
- An LED, acting as an indicator, was connected to digital pin 9 through a 220 Ω resistor.
- The Arduino board was powered and interfaced with a computer via a USB cable, which established the serial communication link for real-time data visualization in Python.

This setup allowed the potentiometer to control the servo's position proportionally while the LED provided a visual signal for threshold crossing and Python displayed the live data graph.

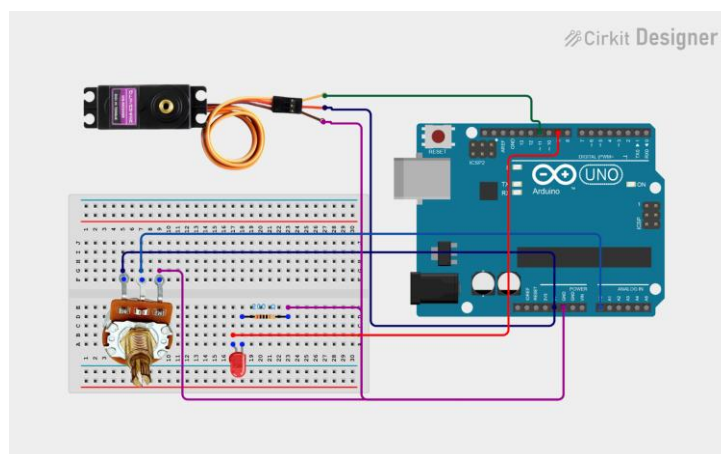


Figure 3.1 Circuit diagram for Task 2

4.0 METHODOLOGY

1. Circuit Construction:

The hardware setup was assembled on a breadboard with the Arduino Uno functioning as the main control unit. A servo motor was interfaced with the Arduino by connecting its control signal wire to digital pin 11, while the power and ground connections were attached to the 5V and GND terminals respectively. A potentiometer was incorporated into the circuit as an adjustable input device, where its outer terminals were connected to the 5V supply and GND, and the center terminal was linked to analog pin A0. Additionally, an LED connected in series with a 220 Ω resistor was attached to digital pin 9 to act as a visual indicator. The Arduino board was connected to a computer through a USB cable, which supplied operating power and established serial communication with the Python application.

2. Programming:

A C++ program was developed and uploaded to the Arduino using the Arduino IDE to manage the servo motor and potentiometer operation. The <Servo.h> library was included to simplify servo control, and the servo motor was initialized on pin 11 using the attach() function. Analog values obtained from the potentiometer through analogRead(A0) were converted into rotational angles between 0° and 180° using the map() function. The calculated angle values were then applied to the servo motor using myservo.write(angle) to control its position dynamically. At the same time, the Arduino transmitted the angle readings to the computer through serial communication using Serial.println(). The LED was programmed to activate whenever the servo angle exceeded a predefined midpoint value and deactivate otherwise. On the computer side, a Python script utilizing the pyserial and matplotlib libraries was executed to receive incoming serial data and display a live graphical visualization of the potentiometer readings.

Arduino Coding:

```
#include <Servo.h>

Servo myservo;
```

```

int potPin = A0;
int ledPin = 9;
int potValue = 0;
int angle = 0;

void setup() {
  Serial.begin(9600);
  myservo.attach(11);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  // Read potentiometer value (0–1023)
  potValue = analogRead(potPin);

  // Map pot value to servo angle (0–180)
  angle = map(potValue, 0, 1023, 0, 180);

  // Move the servo
  myservo.write(angle);

  // LED control: ON if angle > 90
  if (angle > 90) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }

  // Send angle to Python
  Serial.println(angle);

  delay(100);
}

```

Phyton Coding:

```

import serial
import matplotlib.pyplot as plt
import time

```

```

# -- Replace 'COMx' with your Arduino's actual serial port (e.g., 'COM3')
ser = serial.Serial('COM11', 9600)
time.sleep(2) # Wait for Arduino to reset

plt.ion() # Interactive mode for real-time plotting
fig, ax = plt.subplots()
x_vals, y_vals = [], []

print("Plotting real-time servo angles. Press CTRL+C in the terminal to stop.")

try:
    while True:
        raw = ser.readline().decode().strip()

        # Check if the incoming data is a valid number
        if raw.isdigit():
            angle = int(raw)
            print(f'Servo Angle: {angle}°')

            x_vals.append(len(x_vals))
            y_vals.append(angle)

            # Update the plot
            ax.clear()
            ax.set_ylim(0, 180)
            ax.set_title("Real-Time Servo Angle Plot")
            ax.set_xlabel("Samples")
            ax.set_ylabel("Angle (°)")
            ax.plot(x_vals, y_vals)
            plt.pause(0.05)

except KeyboardInterrupt:
    print("\nSending stop command to Arduino...")
    # Send the stop signal to the Arduino
    ser.write(b'x')
    time.sleep(1)
finally:
    ser.close()
    plt.ioff() # Turn off interactive mode
    plt.show() # Display the final static plot
    print("Serial connection closed.")

```

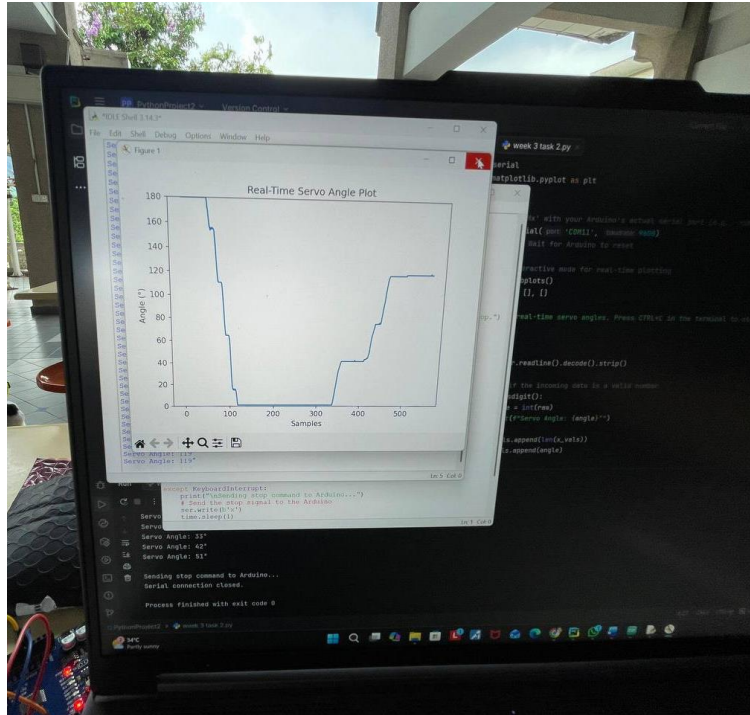


Figure 4.1: Real-time plotted data of Potentiometer using Matplotlib in Python

3. Operation

Once the Arduino program had been uploaded successfully, the system was powered via the USB connection. The Python application was then launched to begin receiving serial data and plotting the readings continuously in real time. As the potentiometer knob was adjusted, the servo motor rotated proportionally according to the changing analog input values. The LED turned ON whenever the servo exceeded the preset threshold angle. Simultaneously, the Python graph updated dynamically to reflect the changing potentiometer values.

4. Observation

During the experiment, the servo motor rotated smoothly across the full range from 0° to 180° in response to the potentiometer adjustment. The LED indicator illuminated when the servo position moved beyond the midpoint angle and switched OFF when the angle decreased below the threshold. The live graph generated in Python displayed immediate and continuous changes

corresponding to the potentiometer movement, indicating stable real-time communication between the Arduino and the computer.

5. Code Behaviour

The Arduino program continuously monitored the analog voltage from the potentiometer and translated the readings into servo rotation angles using the `map()` function. PWM control signals generated through the Servo library allowed accurate positioning of the servo motor. LED control was implemented using conditional logic based on the current servo angle value. On the Python side, the incoming serial data were validated using the `isdigit()` function before being plotted using Matplotlib's interactive plotting mode (`plt.ion()`). Error handling through `KeyboardInterrupt` was also included to ensure that the serial port terminated safely when the program was stopped by the user. This overall setup successfully demonstrated real-time interaction between hardware and software systems involving analog sensing, actuator control, serial communication, and live data visualization.

5.0 DATA COLLECTION

In this experiment, the system was upgraded to provide real-time servo motor control using a potentiometer connected to the Arduino Uno. The Arduino continuously read the potentiometer's analog input values and converted them into servo rotation angles ranging from 0° to 180°. These angle values were then transmitted to the Python application through serial communication for monitoring and visualization purposes.

Potentiometer Reading (0–1023)	Mapped Servo Angle (°)	LED Status	Observations
0	0	OFF	Servo at 0°, LED off
256	45	OFF	Slight servo rotation
512	90	ON	Servo at mid-position, LED turns on
768	135	ON	Servo further rotated
1023	180	ON	Servo fully rotated, LED on

The collected data were displayed and recorded in real time using the Python program, allowing direct observation of the relationship between the potentiometer position and the resulting servo angle. The graphical output generated in Python illustrated an approximately linear relationship between the potentiometer readings and the servo motor rotation.

6.0 DATA ANALYSIS

Analysis of the collected data showed a proportional relationship between the potentiometer input values and the angular position of the servo motor. As the potentiometer resistance varied, the analog readings obtained by the Arduino increased or decreased accordingly. These readings were mapped into servo rotation angles, resulting in corresponding changes in the servo motor position.

The LED threshold condition, configured near the midpoint analog value of 512, provided a simple indication of whether the servo motor had exceeded its central position. The real-time graph produced using Matplotlib demonstrated smooth data transitions and minimal communication delay between the potentiometer adjustment and servo response. This confirmed that the serial communication operating at 9600 baud was sufficiently stable for continuous real-time monitoring and actuator control.

Overall, the experiment demonstrated effective synchronization between analog sensor input, Arduino-based signal processing, servo motor actuation, and Python-based real-time visualization.

7.0 RESULTS

The experiment successfully achieved all objectives. The servo motor responded accurately to potentiometer input transmitted via serial communication, and the relationship between input voltage and servo angle followed a clear linear trend. The integration of the LED indicator provided immediate visual confirmation of system state, while Python's live plotting capability allowed for continuous monitoring of the potentiometer's output.

The following outcomes were recorded:

1. **Successful Serial Communication** – The Arduino and Python programs exchanged data seamlessly at 9600 bps, confirming that serial transmission can handle real-time control signals.
2. **Accurate Servo Motion** – The servo motor consistently rotated to the correct angle corresponding to the potentiometer's position, validating the correctness of the function.
3. **Real-Time Visualization** – The live plotting system updated in real time with minimal lag, reflecting both the hardware and software responsiveness of the system.
4. **Feedback Integration** – The LED control feature effectively indicated servo position thresholds, enhancing user feedback and system interactivity.
5. **System Stability** – The communication remained stable throughout extended operation, showing no data loss or transmission errors.

The overall performance of the system demonstrates the effective integration of analog sensing, digital actuation, and serial data communication within a mechatronic framework. The experiment provides a strong foundation for developing more complex control systems, such as PID-based servo control or multi-sensor automation.

In conclusion, Test 2 validated the principles of real-time serial communication and control using Arduino and Python. It effectively showed how hardware inputs can be processed, transmitted, and visualized through software to achieve responsive and intelligent mechatronic operation.

8.0 DISCUSSION

The experiments successfully demonstrated the practical implementation of serial communication between a microcontroller and a computer in a mechatronic system. The two tests were designed to progressively develop understanding, starting with basic data transmission and advancing toward interactive hardware control. This progression effectively illustrates how sensing, communication, and actuation are integrated in embedded systems.

In the first test, one-way serial communication was established between Arduino and Python. The potentiometer functioned as an analog input device, generating varying voltage levels based on its rotational position. The Arduino's analog-to-digital converter translated these voltages into digital values, which were then transmitted to the computer. The consistent relationship between the potentiometer position and the recorded values confirmed that the communication parameters, including baud rate and data formatting, were correctly configured. This stage provided a reliable foundation for real-time data acquisition and visualization.

The second test extended the system into a more interactive application by incorporating servo motor control and visual feedback. The potentiometer readings were mapped into corresponding servo angles, enabling direct physical movement based on user input. An LED indicator was used to represent a threshold condition, providing additional system feedback. Python-based real-time plotting further enhanced monitoring by displaying continuous changes in the input signal.

Overall, the experiments demonstrated the transition from simple data monitoring to a functional control system. The first test focused on accurate data collection, while the second introduced real-time actuation and feedback, forming a basic closed-loop structure. Minor

issues such as slight response delays highlighted the importance of proper timing and synchronization in serial communication systems.

In conclusion, the results confirm that Arduino–Python serial communication is a reliable method for integrating sensor data processing, actuator control, and real-time visualization. The system performed consistently and accurately, reinforcing its suitability for embedded and mechatronic applications where hardware–software interaction is required.

9.0 CONCLUSION

In conclusion, the serial communication experiments between Arduino and Python were successfully implemented and tested. The experiments demonstrated reliable data transmission between the Arduino microcontroller and the computer through USB serial communication. In Experiment 1, the potentiometer readings were successfully transmitted from Arduino to Python, allowing real-time monitoring of sensor values. Bidirectional communication was also achieved, where Python successfully controlled the LED state based on the potentiometer readings received from Arduino.

In Experiment 2, the servo motor was successfully controlled through serial commands transmitted from Python to Arduino. The servo responded accurately to the input angle values, demonstrating effective communication and control between the software and hardware systems. In addition, real-time data visualization using Python and Matplotlib was successfully implemented, allowing continuous monitoring of servo angle changes graphically.

The experiment achieved all objectives by demonstrating the integration of microcontrollers, sensors, actuators, serial communication, and Python programming in a mechatronic control system. The results validated the effectiveness of serial communication as a simple and reliable method for real-time hardware monitoring and control applications. Overall, the experiment provided valuable understanding of Arduino-Python interfacing, sensor data acquisition, actuator control, and real-time data visualization in mechatronic systems.

10.0 RECOMMENDATIONS

- **Improve Serial Communication Speed**

Higher baud rates can be used to improve the speed of data transmission between Arduino and Python, resulting in faster system response and smoother real-time monitoring.

- **Add More Sensors and Actuators**

The system can be expanded by integrating additional sensors such as temperature, ultrasonic, or motion sensors together with extra actuators for more advanced automation applications.

- **Develop a Graphical User Interface (GUI)**

A dedicated GUI can be developed using Python libraries such as Tkinter or PyQt to provide a more user-friendly interface for monitoring sensor data and controlling hardware devices.

- **Optimize Servo Motor Control**

Smoother servo movement can be achieved by improving angle mapping and reducing signal delay in the control program.

11.0 REFERENCES

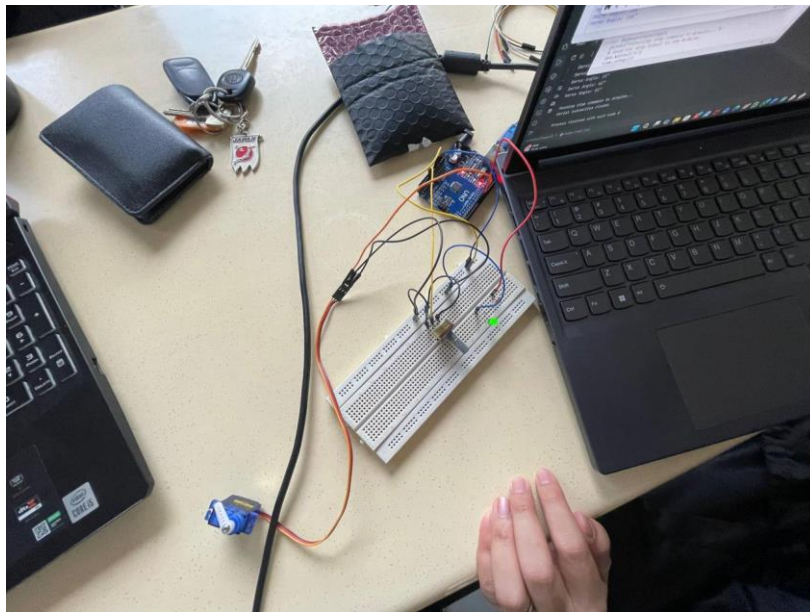
Instructables. (2015, July 10). *Arduino Servo Motors*. Instructables; Instructables.

<https://www.instructables.com/Arduino-Servo-Motors/>

Last Minute Engineers. (2019, October 20). *How Servo Motor Works & Interface It With Arduino*. Last Minute Engineers. [https://lastminuteengineers.com/servo-motor-](https://lastminuteengineers.com/servo-motor-arduino-tutorial/)

[arduino-tutorial/](https://lastminuteengineers.com/servo-motor-arduino-tutorial/)

APPENDICES



ACKNOWLEDGEMENTS

The group would like to express our deepest appreciation to our laboratory instructor, Dr. -Ig. Wahju Sediono and Assoc. Prof. Eur. Ing. Ir. Ts. Gs. Inv. Dr. Zulkifli bin Zainal Abidin for the continuous guidance, encouragement, and clear explanations provided throughout the course of this experiment. His expertise and detailed feedback greatly enhanced our understanding of digital logic systems and circuit interfacing, especially in applying theoretical knowledge to practical implementation. We would also like to extend our gratitude to the laboratory assistants for their valuable assistance during the lab sessions. Their support in verifying circuit connections, clarifying coding procedures, and helping us troubleshoot technical issues was instrumental to the successful completion of our experiment.

In addition, we would like to thank our classmates and also group members, Anis Aqilah Binti Abdullah, Amir Aiman bin Ariff Affendi and Azril Ulwan Bin Mohammad Imri for their cooperation and willingness to share insights, suggestions, and resources during the experiment. Collaborative discussions and teamwork played a crucial role in identifying and resolving problems efficiently. Lastly, we appreciate the opportunity provided by the Department of Mechatronics Engineering to conduct this experiment, which allowed us to strengthen our practical skills in Arduino programming, electronic interfacing, and logical circuit design. The collective guidance and support from all parties involved contributed significantly to the success and learning outcome of this project.

STUDENTS' DECLARATION

Certificate of Originality and Authenticity

This is to certify that we are **responsible** for the work submitted in this report, that **the original work** is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report **has not been done by only one individual**, and **all of us have contributed to the report**. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have **read** and **understand** the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for **marking**, and this **final printed report** has been **verified by us**.

Signature:  Name: Amir Aiman Bin Ariff Affendi Matric Number: 2315925 Contributions: Introduction, Materials and Equipment, Experimental Setup, Results	Read [/] Understand [/] Agree [/]
Signature:  Name: Anis Aqilah Binti Abdullah Matric Number: 2329678 Contributions: Methodology, Data Collection, Data Analysis, Discussions	Read [/] Understand [/] Agree [/]
Signature:  Name: Azril Ulwan Bin Mohammad Imri Matric Number: 2314617 Contributions: Conclusion, Recommendations, References, Appendices, Acknowledgements	Read [/] Understand [/] Agree [/]